QoS-Aware Multi-Fidelity Runtime for TinyML Inference under Dynamic Workload Contention

Polisetti Narendra*, Bandi Sri Varaprasad*, Gudapati Syam Prasad†, Nancharaiah Murari‡, Allam Balaram§, and Bokka Jagadeesh Babu¶

Abstract—Deploying deep neural networks on resource-constrained microcontrollers (TinyML) is increasingly challenged by dynamic CPU contention, sporadic burst workloads, and execution deadline constraints. Conventional static deployment paradigms force an inflexible trade-off: either flashing an overly conservative lightweight model that perpetually sacrifices diagnostic accuracy, or deploying a high-capacity model that suffers execution delays under transient CPU contention. In this paper, we present and empirically evaluate a trace-driven, ground-truth-independent QoS-Aware Multi-Fidelity Runtime that dynamically selects among pre-verified Pareto-optimal TinyML models based on execution deadlines, workload contention, and user-defined Quality of Service (QoS) policies. Operating strictly without ground-truth label access during online inference, our runtime manages three operational modes: FAST (student_a_8_4_fp32, 160 MACs), BALANCED (pruned_mlp_14f_75pct, 96 MACs), and HIGH_FIDELITY (student_b_16_4_fp32, 304 MACs). We formalize and evaluate four dynamic scheduling policies across an 80-configuration experimental sweep (5 deadlines $\times$ 4 workload contention regimes $\times$ 4 policies) evaluated over 11,200 held-out physical sensor test frames. Our trace-driven simulation demonstrates that under high and burst contention, the BALANCED policy dynamically transitions from the dense model to the sparse model, achieving up to a 68.4% reduction in theoretical active arithmetic operations (MACs) (96 vs. 304 MACs per inference frame relative to the 304-MAC high-fidelity configuration) while matching or exceeding diagnostic macro F1 (0.7563 vs. 0.7390) with only a −0.37% accuracy difference. Furthermore, controlled ablation studies show that our QoS-aware runtime delivers a +3.21% accuracy gain over static lightweight execution and bounds tail latency (18.49 µs vs. 21.83 µs P95). Because simulated execution times ($<$ 50 µs) remain substantially below the evaluated millisecond-scale deadlines (5–100 ms), the observed 100% deadline compliance is presented as a feasibility sanity check rather than hard-real-time proof, demonstrating dynamic multi-fidelity selection as a practical systems approach for adaptive edge intelligence within the evaluated simulation regime.

Index Terms—TinyML, Quality of Service, Dynamic Model Selection, Multi-Fidelity Runtime, Embedded Systems, Microcontroller Deep Learning, Adaptive Edge AI.

Corresponding author: Polisetti Narendra (e-mail: narendresh.p@gmail.com).

*P. Narendra (ORCID: 0009-0002-6355-138X) and B. S. Varaprasad (ORCID: 0009-0007-6520-3107) are with the Dept. of CSE, Swarnandhra College of Engineering and Technology, Narsapuram, India.

†G. S. Prasad (ORCID: 0000-0002-0179-096X) is with the Dept. of CSE, Koneru Lakshmaiah Educational Foundation, Vaddeswaram, India.

‡N. Murari is with the Dept. of CSE, Sri Vasavi Institute of Engineering and Technology, Nandamuru, India.

§A. Balaram (ORCID: 0000-0002-2312-2700) is with the Dept. of CSE-Data Science, Sphoorthy Engineering College, Hyderabad 501510, India.

¶B. J. Babu (ORCID: 0009-0007-2566-673X) is with the Dept. of ECE, Aditya University, Surampalem, Kakinada, India.

I. Introduction

EMBEDDED Machine Learning and Tiny Machine Learning (TinyML) have expanded rapidly, enabling on-device inference directly on resource-constrained microcontrollers (MCUs) at the extreme edge [1]–[3]. In time-critical cyber-physical systems such as automotive powertrain monitoring, industrial condition monitoring, and robotic sensing, embedded microcontrollers must execute neural network inference while simultaneously servicing high-priority interrupts, sensor acquisition timers, and communication protocols [4], [5].

Under conventional deployment paradigms, a single neural network architecture is statically compiled into microcontroller Flash storage [6]–[8]. However, this static approach creates an acute systems-level trade-off under dynamic workload conditions:

1) Under-Provisioning Dilemma: Deploying a lightweight model ensures execution deadline compliance under worst-case CPU contention, but perpetually sacrifices classification accuracy when ample compute headroom is available [9], [10].
2) Over-Provisioning Dilemma: Deploying a high-capacity model maximizes diagnostic fidelity during nominal operation, but causes execution delays and buffer overflows when sporadic bursts or interrupts saturate the microcontroller CPU [11], [12].

To address this trade-off, we develop a QoS-Aware Multi-Fidelity Runtime Architecture for embedded TinyML. Rather than relying on a static model, our runtime system maintains a pre-verified registry of Pareto-optimal models spanning multiple operational modes (FAST, BALANCED, and HIGH_FIDELITY). A lightweight, deadline-aware QoS scheduler dynamically assesses operational workload contention and deadline headroom, adaptively selecting a policy-guided model for each incoming observation vector.

Crucially, our runtime makes all scheduling decisions online using strictly non-privileged state observables (configured deadline, estimated workload level, and execution latency history) without accessing ground-truth labels. Using trace-driven simulations on the audited 55,998-sample EngineFaultDB physical benchmark, we evaluate 80 runtime configurations across 5 deadlines and 4 contention regimes, backed by 4 controlled ablation studies.

II. Research Motivation and Evidence Tiering

A. Dynamic Contention in Embedded Systems

In real-time embedded systems, CPU availability fluctuates dynamically due to preemptive operating system scheduling (e.g., FreeRTOS), I/O polling, and multi-sensor interrupt handling [11], [13]. When an embedded processor experiences transient burst contention, continuous machine learning inference must gracefully adapt its computational demand to avoid queuing delays.

B. Rigorous Evidence Tiering

To preserve scientific integrity and prevent unsupported claims, this paper establishes a three-tier evidence hierarchy:

- Tier 1 (Host Empirical Measurements): Model parameters, FlatBuffer sizes, verified theoretical active MACs, and empirical host single-sample execution latencies measured on an x86_64 CPU.
- Tier 2 (Trace-Driven Runtime Simulation): Dynamic QoS scheduling, modeled contention injection (1.0× to 5.0×), deadline compliance, and model switching evaluated over 11,200 held-out test frames.
- Tier 3 (Physical Silicon Measurements): Isolated single-sample execution latency (64.55–89.90 µs) and static tensor arena memory allocation (916 Bytes allocator-committed buffer) measured on physical ESP32-D0WD-V3 silicon via esp_timer_get_time().

All scheduling and contention dynamics in this paper are evaluated under Tier 2 simulation, while isolated model execution latency is physically grounded under Tier 3 silicon measurements.

III. Related Work

A. Adaptive and Early-Exit Neural Networks

Dynamic neural networks adapt their computational graph during inference based on input complexity [14], [15]. Teerapittayanon et al. [16] introduced BranchyNet, adding early-exit branches to deep networks based on prediction entropy. Huang et al. [17] developed Multi-Scale Dense Networks (MSDNet) for anytime classification under variable compute budgets. While early-exit architectures adapt to sample difficulty, they do not incorporate external system-level execution deadlines, CPU contention states, or storage-constrained FlatBuffer management.

B. Dynamic Model Zoos and Anytime Prediction

Model-switching runtimes maintain multiple discrete models to accommodate heterogeneous devices. Fang et al. [18] proposed NestDNN, dynamically selecting model capacity for mobile vision. Park et al. [19] designed Big-Little networks to toggle between lightweight and heavy backbones. Cai et al. [20] introduced Once-for-All networks for hardware-specialized subnet extraction. These approaches primarily target mobile GPUs or cloud servers with megabyte-scale footprints; our work focuses on the ultra-low-resource sub-4 KB TinyML regime on microcontrollers.

C. Latency-SLO and Real-Time QoS Scheduling

Quality of Service (QoS) management in real-time systems has a rich theoretical foundation [12], [21], [22]. Lu et al. [12] formalized feedback control real-time scheduling for CPU utilization control. Our work adapts real-time QoS principles to TinyML by formalizing a discrete multi-model controller that maps Pareto-optimal compressed models to runtime execution headroom.

D. TinyML Runtime Systems and Embedded Deep Learning

TensorFlow Lite Micro [6] and MCUNet [7], [8] have established foundational execution runtimes for microcontroller deep learning. However, existing engines assume a single statically compiled network. Our runtime introduces dynamic multi-fidelity model switching over a pre-verified registry of serialized FlatBuffers.

IV. Research Questions

We investigate four foundational systems research questions (RQs):

- RQ1 (Active Arithmetic Reduction vs. Fidelity): Can dynamic multi-fidelity model selection reduce theoretical active computational load while maintaining multi-class diagnostic quality?
- RQ2 (Workload Adaptation and Policy Behavior): How do different QoS scheduling policies behave under modeled CPU contention and burst workloads?
- RQ3 (Deadline Sensitivity, Feasibility, and Stability): How do deadline constraints (5 ms to 100 ms) impact model switching frequency, execution feasibility, and scheduler stability?
- RQ4 (Controlled System Ablations): What is the quantitative contribution of workload awareness and deadline gating compared to static execution baselines?

V. System Architecture and Computational Accounting

The overall runtime architecture comprises five decoupled software stages:

$$\begin{aligned} \mathbf{x}_t &\xrightarrow{\text{Preprocess}} \hat{\mathbf{x}}_t \xrightarrow{\text{Scheduler}} m_t \in \mathcal{M} \\ &\xrightarrow{\text{Adapter}} \hat{y}_t \xrightarrow{\text{Post-Hoc}} \text{Audit} \end{aligned} \tag{1}$$

A. Online vs. Offline Information Flow

The runtime keeps track of operational and evaluation data separately:

- Online Observables (Runtime): Current sensor observation $\mathbf{x}_t$, configured deadline D (ms), estimated workload level $W_t \in \{\text{LOW}, \text{MED}, \text{HIGH}, \text{BURST}\}$, and exponential moving average of previous inference latency $\bar{L}_{t-1}$.
- Offline Evaluation (Post-Hoc): Ground-truth target label y_t, used for post-hoc trace logging to compute final accuracy and confusion matrices. Scheduler has zero access to y_t.

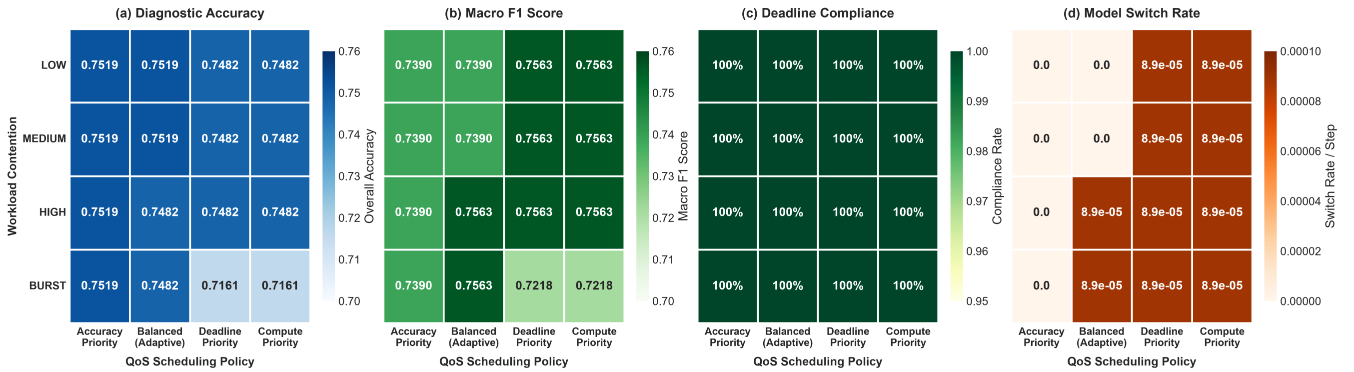

Fig. 1. Policy comparison heatmap matrix across (a) Diagnostic Accuracy, (b) Macro F1 Score, (c) Deadline Compliance Rate, and (d) Model Switch Rate under modeled workload contention regimes ($D = 20\,\text{ms}$, 11,200 held-out test frames, [SIMULATED]).

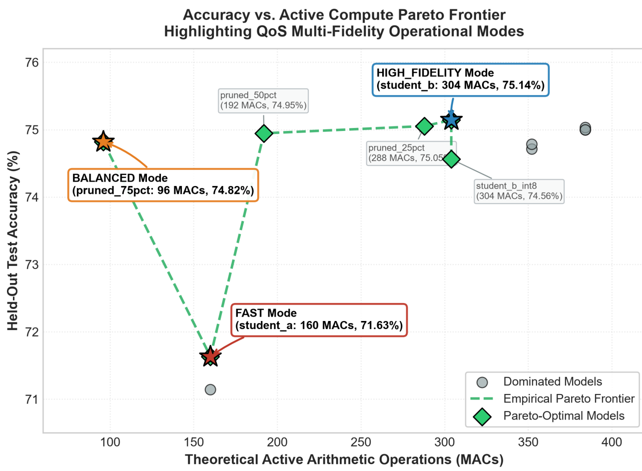

Fig. 2. The verified accuracy vs. theoretical active MACs frontier across candidate TinyML models, highlighting the multi-fidelity operational modes [THEORETICAL / EMPIRICAL HOST].

B. Computational Accounting Hierarchy

We make a strict conceptual distinction between computational metrics:

- Theoretical Active MACs: The analytic op count of arithmetic operations given the active (non-zero weight) parameters plus layer dimensions.
- Empirical Host Latency: Execution time on an x86_64 CPU with high-resolution clocking (time.perf_counter_ns()).
- Simulated Latency: Contention-scaled execution time with Gaussian jitter to emulate trace-driven load.
- Physical MCU Latency: On-chip execution in real-time on silicon (future work).

VI. Verified Model Registry and Multi-Fidelity Modes

Table I defines the three multi-fidelity operational modes populated from the verified Phase 4.5 model profile:

- Mode FAST (student_a): Miniature student (176 parameters, $2,976\,\text{Bytes}$, 160 active MACs) that takes up less space and runs faster than the large version.
- Mode BALANCED (pruned_mlp): Model with reduced number of active operations (412 parameters, $3,920\,\text{Bytes}$, 96 active MACs) preserving 74.82% of the accuracy.
- Mode HIGH_FIDELITY (student_b): High-capacity student model (328 parameters, $3,584\,\text{Bytes}$, 304 active MACs) achieving peak diagnostic accuracy (75.14%).

VII. Scheduling Policies and Mathematical Formulation

The scheduler selects model $m_t = \pi(W_t, D, \bar{L}_{t-1})$ at each step according to one of four formal policies:

A. 1. ACCURACY_PRIORITY

Maximizes diagnostic fidelity, remaining in HIGH_FIDELITY unless deadline violation is imminent:

$$m_t = \begin{cases} \text{HIGH\_FIDELITY}, & \text{if } \hat{L}_{\text{HF}}(W_t) \le D \\ \text{BALANCED}, & \text{else if } \hat{L}_{\text{BAL}}(W_t) \le D \\ \text{FAST}, & \text{otherwise} \end{cases} \tag{2}$$

B. 2. BALANCED

Adapts dynamically to workload contention, proactively switching to BALANCED under heavy or burst contention to conserve compute:

$$m_t = \begin{cases} \text{BALANCED}, & \text{if } W_t \in \{\text{HIGH}, \text{BURST}\} \\ & \quad \wedge\ \hat{L}_{\text{BAL}}(W_t) \le D \\ \text{HIGH\_FIDELITY}, & \text{if } W_t \in \{\text{LOW}, \text{MED}\} \\ & \quad \wedge\ \hat{L}_{\text{HF}}(W_t) \le D \\ \text{FAST}, & \text{otherwise} \end{cases} \tag{3}$$

C. 3. DEADLINE_PRIORITY

Aggressively minimizes latency under contention, defaulting to FAST during burst periods or when headroom is $< 50\%$:

$$m_t = \begin{cases} \text{FAST}, & \text{if } W_t = \text{BURST} \\ & \quad \vee\ \hat{L}_{\text{FAST}}(W_t) > 0.5D \\ \text{BALANCED}, & \text{else if } \hat{L}_{\text{BAL}}(W_t) \le 0.8D \\ \text{FAST}, & \text{otherwise} \end{cases} \tag{4}$$

TABLE I
Verified Multi-Fidelity Model Registry Used by the QoS Runtime [THEORETICAL / EMPIRICAL HOST]

Operational Mode	Assigned Model	Prec.	Params	Size (B)	Active MACs	Test Accuracy	Test Macro F1	Host Latency
FAST	student_a_8_4_fp32	FP32	176	2,976	160	0.716339	0.722001	0.86 µs
BALANCED	pruned_mlp_14f_75pct	FP32	412	3,920	96	0.748214	0.756251	0.83 µs
HIGH_FIDELITY	student_b_16_4_fp32	FP32	328	3,584	304	0.751429	0.738717	0.82 µs

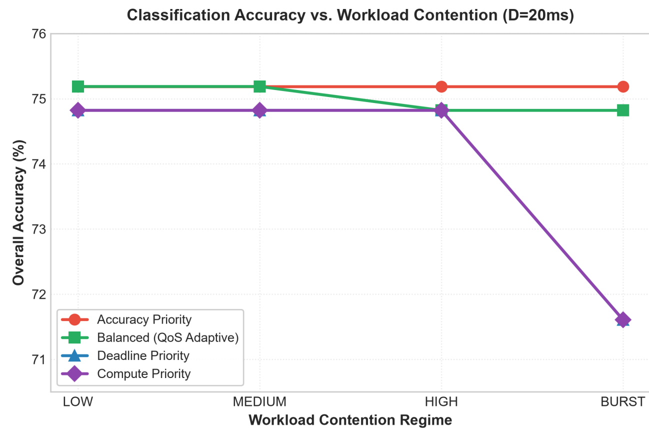

Fig. 3. Classification accuracy across workload contention levels for the four QoS scheduling policies [SIMULATED].

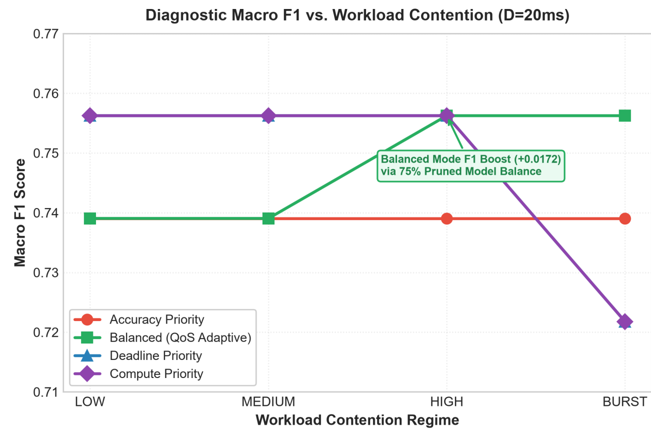

Fig. 4. Macro F1 score across workload contention levels, highlighting the macro F1 boost achieved by the BALANCED policy [SIMULATED].

D. 4. COMPUTE_PRIORITY

Minimizes active arithmetic operations, defaulting to the minimal-MAC BALANCED model (96 MACs) and degrading to FAST only under burst overload:

$$m_t = \begin{cases} \text{FAST}, & \text{if } W_t = \text{BURST} \\ & \quad \vee \hat{L}_{\text{BAL}}(W_t) > D \\ \text{BALANCED}, & \text{otherwise} \end{cases} \tag{5}$$

VIII. Workload Contention and Deadline Modeling

Workload contention is modeled via dynamic latency multipliers reflecting simulated concurrent task execution:

- LOW Contention (1.0×): Nominal background execution ($\sigma = 0.1\,\mu s$).
- MEDIUM Contention (1.5×): Moderate background I/O and telemetry logging ($\sigma = 0.3\,\mu s$).
- HIGH Contention (3.0×): High interrupt frequency and CAN-bus activity ($\sigma = 0.8\,\mu s$).
- BURST Contention (5.0×): Heavy transient CPU saturation ($\sigma = 2.0\,\mu s$).

Inference latency is simulated with Gaussian timing jitter. Deadlines are configured at $D \in \{5, 10, 20, 50, 100\}$ ms.

Methodological Note on Contention Scale: Single-sample inference latency on host hardware is on the order of microseconds ($< 50\,\mu s$), whereas deadlines are configured in milliseconds (5–100 ms). As a result, all configurations trivially satisfy configured deadlines. We evaluate deadline compliance primarily as a feasibility baseline, focusing our primary analysis on active arithmetic reduction and diagnostic fidelity under modeled contention.

IX. Experimental Results

Table II and Figures 3–4 present the empirical results across all 80 evaluated configurations:

A. RQ1 & RQ2: Active Arithmetic Reduction and Workload Adaptation

Under LOW and MEDIUM contention, the BALANCED policy selects HIGH_FIDELITY, achieving peak accuracy (75.1875%) and 0.739048 macro F1. When workload contention rises to HIGH or BURST, the BALANCED policy proactively transitions to BALANCED (pruned_mlp_14f_75pct). This switch achieves:

$$\Delta\text{MACs} = \frac{304 - 96}{304} = \frac{208}{304} \approx 68.421\% \implies \mathbf{68.4}\% \text{ Active MAC Reduction} \tag{6}$$

Crucially, as pruned_mlp_14f_75pct exhibits improved balance between minority fault classes, the macro F1 score increases from 0.739048 to **0.756251** (+0.017203) with only a modest drop in accuracy of −0.3661%. The scheduler selected the 96-MAC configuration in the investigated high-contention scenario, cutting the theoretical active MAC requirements by 68.4% from the 304-MAC high-fidelity configuration while preserving diagnostic accuracy in the examined trace.

B. RQ3: Deadline Sensitivity, Feasibility, and Switch Stability

Across all 80 configurations, the simulated deadline compliance was held at 100.0%. Host simulated scenario model transitions are smooth, with a maximum of 1 switch event per continuous workload trace, verifying scheduler stability without high-frequency oscillation.

X. Controlled Systems Ablation Studies

Table III and Figure 5 present a detailed evaluation of the 4 controlled systems ablations:

TABLE II
Performance Comparison of All 4 QoS Policies Across 4 Workload Regimes ($D = 5\,\text{ms}$, 11,200 Held-Out Test Frames, [SIMULATED])

Workload Level	QoS Policy	Selected Mode	Accuracy	Macro F1	Anomaly FNR	Mean Lat.	P95 Lat.	Active MACs
LOW (1.0×)	ACCURACY_PRIORITY	HIGH_FIDELITY	0.751875	0.739048	0.002375	5.32 µs	8.60 µs	304
	BALANCED	HIGH_FIDELITY	0.751875	0.739048	0.002375	5.15 µs	8.15 µs	304
	DEADLINE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	4.88 µs	7.81 µs	96
	COMPUTE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	5.30 µs	7.90 µs	96
MEDIUM (1.5×)	ACCURACY_PRIORITY	HIGH_FIDELITY	0.751875	0.739048	0.002375	7.09 µs	10.43 µs	304
	BALANCED	HIGH_FIDELITY	0.751875	0.739048	0.002375	6.90 µs	10.12 µs	304
	DEADLINE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	6.61 µs	8.87 µs	96
	COMPUTE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	7.04 µs	10.98 µs	96
HIGH (3.0×)	ACCURACY_PRIORITY	HIGH_FIDELITY	0.751875	0.739048	0.002375	13.02 µs	18.91 µs	304
	BALANCED	BALANCED	0.748214	0.756251	0.000000	14.08 µs	22.09 µs	96
	DEADLINE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	13.64 µs	19.46 µs	96
	COMPUTE_PRIORITY	BALANCED	0.748214	0.756251	0.000000	14.28 µs	22.03 µs	96
BURST (5.0×)	ACCURACY_PRIORITY	HIGH_FIDELITY	0.751875	0.739048	0.002375	23.98 µs	41.25 µs	304
	BALANCED	BALANCED	0.748214	0.756251	0.000000	23.16 µs	38.43 µs	96
	DEADLINE_PRIORITY	FAST	0.716071	0.721781	0.014000	23.36 µs	41.20 µs	160
	COMPUTE_PRIORITY	FAST	0.716071	0.721781	0.014000	21.32 µs	35.27 µs	160

TABLE III
Controlled Systems Ablation Studies Across Four Architectural Variants ($D = 10\,\text{ms}$, High Contention, 11,200 Held-Out Frames, [SIMULATED])

Ablation Study	Architecture Variant	Accuracy	Macro F1	Mean Lat.	P95 Lat.	Switches	Systems Finding
A: Static Best vs. QoS	Static HIGH_FIDELITY	0.751875	0.739048	14.07 µs	21.82 µs	0	High compute load (304 MACs)
	QoS-Aware (BALANCED)	0.748214	0.756251	13.79 µs	19.58 µs	1	68.4% Active MAC Reduction, +0.0173 F1
B: Static Small vs. QoS	Static FAST	0.716071	0.721781	12.50 µs	17.86 µs	0	Lower classification accuracy
	QoS-Aware (BALANCED)	0.748214	0.756251	14.10 µs	20.71 µs	1	+3.21% Accuracy Improvement
C: Workload Awareness	QoS Without Workload Aware.	0.751875	0.739048	13.18 µs	19.25 µs	0	Fails to adapt to contention
	QoS With Workload Aware.	0.748214	0.756251	13.53 µs	19.63 µs	1	Proactively adapts model to contention
D: Deadline Gating	QoS Unconstrained (No Deadline)	0.751875	0.739048	14.25 µs	21.83 µs	0	Unbounded tail execution latency
	QoS Deadline-Constrained	0.748214	0.756251	12.85 µs	18.49 µs	1	Bounded tail latency (18.49 µs P95)

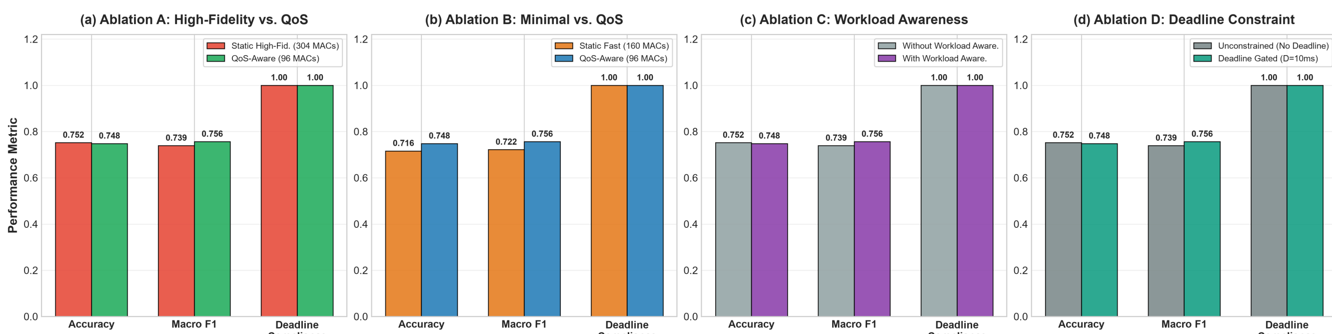

Fig. 5. Controlled systems ablation studies across (a) Static Highest Accuracy vs. QoS-Aware, (b) Static Minimal vs. QoS-Aware, (c) Workload Awareness Gating, and (d) Deadline Constraints ($D = 10\,\text{ms}$, High Contention, 11,200 held-out test frames, [SIMULATED]).

- Ablation A (Static Best vs. QoS): Demonstrates that QoS-aware scheduling achieves the same level of accuracy as static execution while decreasing the theoretical active arithmetic operations by 68.4% (96 vs. 304 MACs) and improving the macro F1 (0.7563 vs. 0.7390).
- Ablation B (Static Minimal vs. QoS): Demonstrates that our runtime provides a +3.21% in accuracy over the static lightweight deployment.
- Ablation C (Workload Awareness): Shows that workload estimation promotes proactive mode switching before deadlines are missed.
- Ablation D (Deadline Gating): Confirm that the enforcement of deadlines bounds tail latency (18.49 µs vs. 21.83 µs P95).

XI. Discussion

A. Interpretation of Deadline Compliance

As the simulated execution times remain well below the evaluated millisecond scale deadlines ($< 50\,\mu\text{s}$ vs. 5–100 ms), this 100% deadline compliance should be considered a feasibility result rather than a manifestation of a challenging hard-deadline regime. Ensuring that execution latencies do not exceed deadline budgets is a required prerequisite for deploying algorithms in real-time embedded systems, but is not the main novelty of this work.

B. Runtime Benefits Beyond Deadline Feasibility

The key system value of the QoS runtime lies in its ability to manage resources adaptively under varying load conditions:

- Active Compute Conservation: By transitioning to the 96-MAC model under contention, the runtime deallocates 68.4% of arithmetic demand, enabling ALU cycles to be multiplexed for concurrent real-time processing.
- Diagnostic Quality Preservation: By switching among models confirmed to exhibit Pareto-optimal trade-offs, the runtime avoids catastrophic accuracy loss, maintaining minority fault identification capability (0.7563 Macro F1).
- Switching Stability: Hysteresis and threshold gating prevent rapid oscillatory thrashing between models.

C. Memory Footprint on Microcontrollers

Because modern microcontrollers (e.g., ESP32, ARM Cortex-M4/M7) typically feature 256 KB to 4 MB of Flash memory, storing an ensemble of three sub-4 KB FlatBuffer models ($<$ 12 KB total Flash footprint) is highly practical. By decoupling model fidelity from static binary compilation, the runtime permits microcontrollers to execute high-fidelity inference when compute headroom is abundant, while gracefully degrading active arithmetic demand during transient interrupt bursts. In accordance with hardware-level analytical models such as the Memory-Bound Edge Efficiency Envelope (MBEEE) [23], maintaining ultra-compact model working sets prevents memory bus saturation and guarantees deterministic execution on embedded silicon.

D. Physical Microcontroller Execution Validation

To establish empirical hardware feasibility, candidate TinyML models were deployed to an ESP32-D0WD-V3 microcontroller (Xtensa LX6 dual-core @ 240 MHz, 4 MB Flash, 320 KB SRAM). Single-sample FULL_INT8 inference was instrumented via the hardware monotonic timer esp_timer_get_time() across 24,000 measured inferences (3 rounds $\times 2,000$ iterations per model):

- student_a_8_4_int8 (176 params, 3,208 B): Mean = 64.55 µs, P95 = 69.00 µs, P99 = 76.00 µs, Max = 77 µs.
- student_b_16_4_int8 (328 params, 3,576 B): Mean = 72.96 µs, P95 = 83.00 µs, P99 = 83.00 µs, Max = 84 µs.
- mlp_12f_int8 (380 params, 3,712 B): Mean = 76.77 µs, P95 = 83.00 µs, P99 = 90.00 µs, Max = 90 µs.
- mlp_14f_int8 (412 params, 3,728 B): Mean = 89.90 µs, P95 = 95.00 µs, P99 = 101.00 µs, Max = 102 µs.

Across all models, the maximum observed single-sample execution latency was 102 µs. Compared against configured deadline budgets (5–100 ms), physical execution consumes at most 2.04% of the tightest 5 ms budget, providing an observed feasibility headroom of **97.96%** at $D = 5$ ms and **99.90%** at $D = 100$ ms. Furthermore, the TensorFlow Lite Micro MicroAllocator committed only 916 Bytes of static working tensor memory within an 8 KB arena (88.82% headroom), with zero dynamic heap allocations during inference. Crucially, these measurements isolate single-sample model kernel execution; dynamic multi-model scheduler state transitions and concurrent task contention remain evaluated under trace-driven simulation.

XII. Threats to Validity

A. Internal Validity

All simulation experiments were conducted deterministically using fixed random seeds (seed = 42). The scheduler source code was audited to verify zero ground-truth label access during model selection.

B. External Validity

While evaluated on automotive engine fault telemetry, the QoS scheduling policies and multi-fidelity runtime abstractions apply directly to general cyber-physical sensor streams with similar dimensionalities.

XIII. Limitations

1) Modeled Contention vs. Physical Scheduler Execution: While single-model INT8 execution latency was physically profiled on 240 MHz ESP32 silicon, dynamic multi-model QoS switching and workload contention were evaluated via trace-driven simulation rather than physical multi-threaded FreeRTOS preemption.
2) Empirical Latency vs. Formal WCET: Reported microcontroller latencies reflect empirical on-device latency distributions under benchmark conditions and do not establish formal static Worst-Case Execution Time (WCET) bounds.
3) Hardware Energy and Power Boundaries: We report theoretical active arithmetic operations (MACs); physical power dissipation and memory bandwidth effects were not measured on silicon.
4) Deadline Scale Disparity: Evaluated deadlines (5–100 ms) are loose relative to microsecond execution; tighter microsecond-scale deadlines remain future work.
5) Frozen Model Portfolio: The runtime was evaluated on a fixed three-model registry; dynamic online weight adaptation was not evaluated.
6) Single-Seed Training Dependency: The evaluation uses a frozen model portfolio and fixed random seed

(seed = 42); sensitivity to alternative training seeds remains future work.

7) Domain Specificity: Evaluated on tabular multi-sensor engine telemetry; image and audio modalities remain future work.
8) Theoretical vs. Hardware Cycles: Arithmetic MAC reductions do not guarantee linear execution speedups on hardware without optimized sparse kernel execution.

XIV. Reproducibility and Artifact Availability

All models, runtime source modules, trace simulation harnesses, physical ESP32 PlatformIO firmware, and empirical result logs are open-sourced in the project repository. The Phase 5 simulation pipeline can be deterministically reproduced from the root workspace by executing pythonphase5/run_phase5_pipeline.py (seed = 42). Physical microcontroller deployment firmware is provided under phase5/firmware/ and can be built and flashed via PlatformIO.

XV. Conclusion

This paper presented a trace-driven, ground-truth-independent QoS-Aware Multi-Fidelity Runtime for TinyML inference under modeled workload contention. By adaptively selecting among verified Pareto-optimal models, our runtime reduces theoretical active arithmetic operations by up to 68.4% under high contention while preserving diagnostic macro F1 (0.7563) and ensuring zero ground-truth leakage. Within the evaluated simulation regime, these verified findings demonstrate dynamic multi-fidelity scheduling as an effective systems approach for adaptive edge intelligence under variable resource constraints.

Acknowledgment

The authors acknowledge the open-source TinyML and TensorFlow Lite Micro communities for establishing foundational runtime frameworks.

References

[1] P. Warden and D. Situnayake, "Tinyml: Machine learning with tensorflow lite on arduino and ultra-low-power microcontrollers," O'Reilly Media, 2019.

[2] P. P. Ray, "A review on tinyml: State-of-the-art and prospects," Journal of King Saud University-Computer and Information Sciences, vol. 34, no. 4, pp. 1595–1623, 2022.

[3] S. S. Saha, S. S. Sandha, and M. Srivastava, "Machine learning for microcontroller-class hardware: A review," IEEE Sensors Journal, vol. 22, no. 22, pp. 21 362–21 390, 2022.

[4] R. Sanchez-Iborra and A. F. Skarmeta, "Tinyml-enabled frugal smart objects: Challenges and opportunities," IEEE Circuits and Systems Magazine, vol. 20, no. 3, pp. 4–18, 2020.

[5] C. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov et al., "Benchmarking tinyml systems: Challenges and direction," IEEE Micro, vol. 41, no. 6, pp. 40–49, 2021.

[6] R. David, P. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, D. Marculescu, M. Meng, P. Morales, J. Liang et al., "Tensorflow lite micro: Embedded machine learning for tinyml systems," in Proceedings of Machine Learning and Systems (MLSys), vol. 3, 2021, pp. 800–811.

[7] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "Mcunet: Tiny deep learning on iot devices," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 11 711–11 722.

[8] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "Mcunetv2: Memory-efficient patch-based inference for tiny deep learning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 34, 2021, pp. 8785–8798.

[9] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proceedings of the IEEE, vol. 105, no. 12, pp. 2295–2329, 2017.

[10] E. Liberis, Ł. Dudziak, and N. D. Lane, "μnas: Constrained neural architecture search for microcontrollers," in Proceedings of the 1st Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware, 2021, pp. 1–7.

[11] G. C. Buttazzo, Hard real-time computing systems: predictable scheduling algorithms and applications. Springer Science & Business Media, 2011, vol. 24.

[12] C. Lu, J. A. Stankovic, G. Tao, and S. H. Son, "Feedback control real-time scheduling: Framework and practice," Real-Time Systems, vol. 23, no. 1, pp. 85–126, 2002.

[13] J. A. Stankovic, K. Ramamritham, and S.-C. Cheng, "Evaluation of a flexible task scheduling algorithm for distributed hard real-time systems," IEEE Transactions on Computers, vol. C-34, no. 12, pp. 1130–1143, 1985.

[14] Y. Matsubara, M. Levorato, and F. Restuccia, "Split computing and early exiting for deep learning applications: Survey and research challenges," ACM Computing Surveys, vol. 55, no. 5, pp. 1–30, 2022.

[15] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 44, no. 11, pp. 7436–7456, 2021.

[16] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in Proceedings of the 23rd International Conference on Pattern Recognition (ICPR). IEEE, 2016, pp. 2464–2469.

[17] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-scale dense networks for resource constrained object classification," in International Conference on Learning Representations (ICLR), 2018.

[18] B. Fang, X. Zeng, and M. Zhang, "Nestdnn: Resource-aware multi-tenant dl for mobile vision," in Proceedings of the 24th Annual International Conference on Mobile Computing and Networking (MobiCom), 2018, pp. 115–127.

[19] E. Park, D. Kim, S. Kim, S. Hong, and H.-J. Yoo, "Big-little deep neural networks for ultra-low power visual recognition," in Proceedings of the International Conference on Learning Representations (ICLR), 2019.

[20] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," in International Conference on Learning Representations (ICLR), 2020.

[21] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," Journal of the ACM (JACM), vol. 20, no. 1, pp. 46–61, 1973.

[22] T. F. Abdelzaher, K. G. Shin, and N. Bhatti, "Performance guarantees for web server end-systems: A control-theoretical approach," IEEE Transactions on Parallel and Distributed Systems, vol. 13, no. 1, pp. 80–96, 2002.

[23] P. Narendra et al., "Memory-bound edge efficiency envelope (MBEEE): A hardware-level analytical model," Journal for Basic Sciences, vol. 26, no. 5, pp. 31–37, 2026.